Instituto Superior Técnico
Organização de Computadores, LEIC-A, 2026/2027

Laboratório 1

System Profiling and Modeling

Grupo 02

André Sá, ist1109904
Carolina Matias, ist1114295
João Varanda, ist1114010

Exercício 1: Comportamento da cache através de medições de tempo

a) Sistema usado

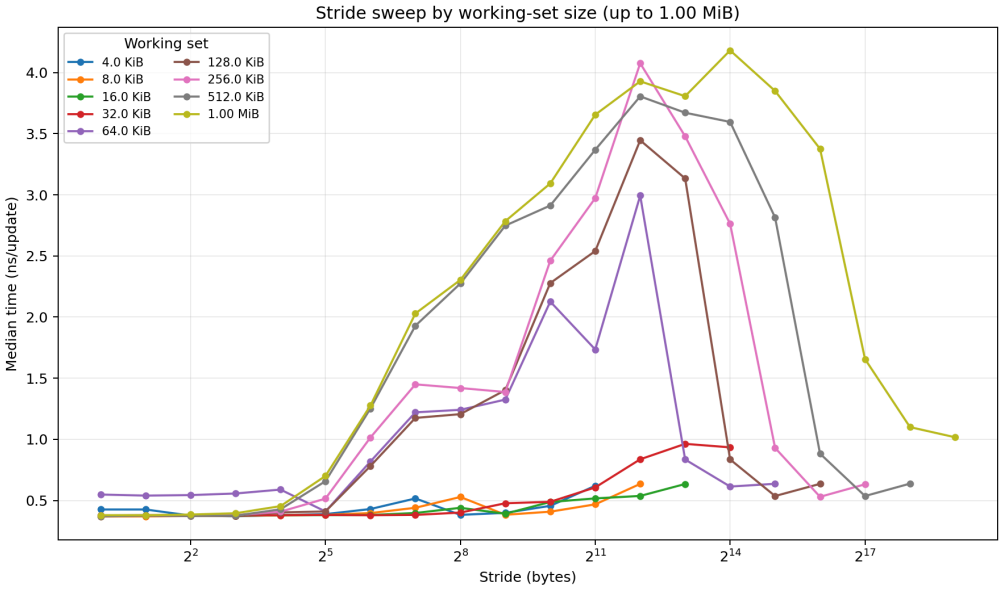
Item	Valor
Processador	Intel(R) Core(TM) i5-10500 CPU @ 3.10GHz
Sistema operativo	Linux 6.12.40, x86_64
Compilador	GCC 14.2.1 20250322
Opções de compilação	`-O2 -std=gnu11 -Wall -Wextra`

Todas as medições do Exercício 1 foram feitas na mesma máquina, com o mesmo compilador e as mesmas opções.

b) Array de 16 KiB - 16384 bytes

Stride (bytes)	Primeiros quatro offsets acedidos	Elementos atualizados por passagem
1	0, 1, 2, 3	16384 / 1 = 16384
32	0, 32, 64, 96	16384 / 32 = 512
64	0, 64, 128, 192	16384 / 64 = 256

c) Capacidade da cache L1 de dados



Estimativa: entre 32 KiB e 64 KiB (intervalo [32, 64[KiB).

Para strides pequenos (1 a 8 B), as curvas estão em cerca de 0,4 ns/update. A exceção é a curva de 64 KiB, a cerca de 0,55 ns. Este valor não se repete na experiência da alínea d), onde o mesmo array dá 0,37 ns. Nestes strides, cada linha carregada serve muitas atualizações consecutivas, e o custo de cada miss fica dividido por essas atualizações, por isso não distinguem os tamanhos. A diferença entre os tamanhos só se torna visível a partir de strides de 64 B:

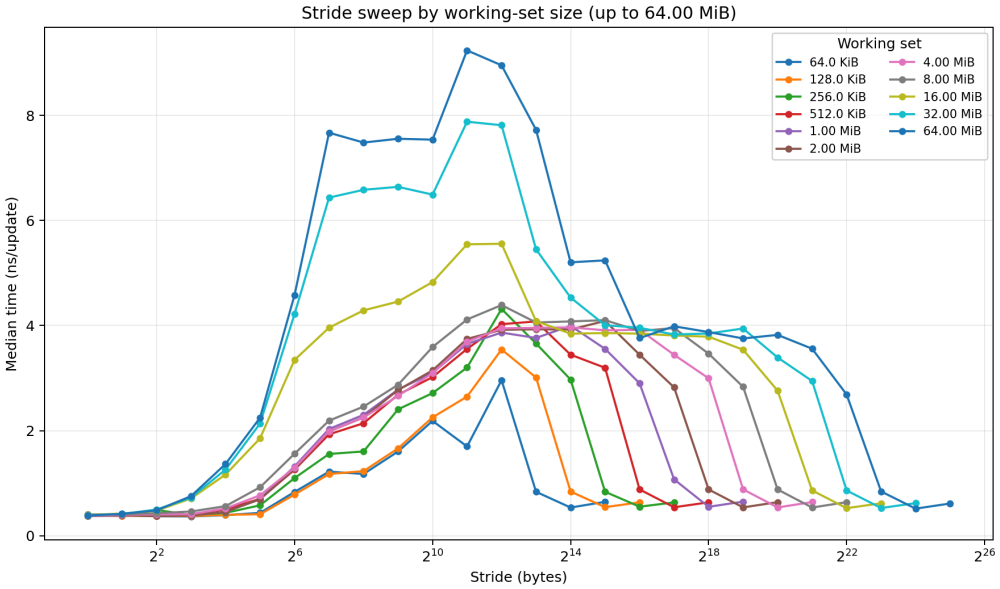
Stride \ Array	4 KiB	8 KiB	16 KiB	32 KiB	64 KiB	128 KiB	256 KiB
64 B	0,43	0,40	0,38	0,38	0,82	0,78	1,02
128 B	0,52	0,44	0,40	0,38	1,22	1,18	1,45
512 B	0,40	0,38	0,39	0,48	1,33	1,40	1,39
1 KiB	0,46	0,41	0,49	0,49	2,13	2,28	2,46

(ns/update, mediana, valores de `e1/L1/cache_table.md`)

As curvas de 4 a 32 KiB ficam todas abaixo de 0,55 ns até strides de 1 KiB. A curva de 64 KiB apresenta uma diferença significativa (0,82 ns com stride de 64 B, 1,22 ns com 128 B e 2,13 ns com 1 KiB) e praticamente coincide com a de 128 KiB. É possível concluir que um array de 32 KiB cabe na L1 e um de 64 KiB já não cabe, ou seja, a capacidade está no intervalo [32, 64[KiB.

d) Capacidade da cache L2

Intervalo de tamanhos usado: mínimo `WORKING_SET_MIN` = 64 KiB, máximo `WORKING_SET_MAX` = 64 MiB. O valor mínimo corresponde ao dobro do limite inferior do intervalo estimado para a capacidade da L1 (32 KiB), garantindo que a primeira curva se situa imediatamente acima desse nível. Os restantes parâmetros do benchmark mantiveram-se inalterados.



Estimativa: cerca de 0,25 MiB (intervalo [0,125; 0,5[MiB).

Stride \ Array	64 KiB	128 KiB	256 KiB	512 KiB	1 MiB	2 MiB	4 MiB
64 B	0,83	0,78	1,10	1,26	1,32	1,28	1,29
128 B	1,22	1,17	1,56	1,93	2,03	1,99	1,99
512 B	1,60	1,66	2,41	2,68	2,78	2,77	2,66
1 KiB	2,19	2,25	2,71	3,02	3,10	3,15	3,10

(ns/update, mediana, valores de `e1/L2/cache_table.md`)

- Entre 64 e 128 KiB as curvas praticamente coincidem, indicando que os dados não excedem a capacidade da L2.
- A partir de 512 KiB observa-se um novo patamar, sustentado e significativamente mais elevado (por exemplo, com stride de 128 B o tempo passa de 1,17 para 1,93 ns, e com 512 B passa de 1,66 para 2,68 ns), que se mantém até 4 MiB.
- O array de 256 KiB apresenta valores intermédios entre os dois patamares, o que indica que os dados excedem parcialmente a capacidade da L2 e situa essa capacidade na proximidade deste valor. As experiências realizadas não permitem determinar a causa do carácter gradual da transição. Uma hipótese é que parte da L2 esteja ocupada por outros dados do programa (por exemplo, código e pilha) e que a política de substituição não seja ótima.

e) Tamanho da linha da L1

Array escolhido: 128 KiB (maior que a L1 estimada, [32, 64[KiB, e menor que a L2 estimada, cerca de 0,25 MiB). O mesmo comportamento observa-se no array de 64 KiB.

Array \ Stride	**1-8 B**	**16 B**	**32 B**	**64 B**	**128 B**	**256 B**	**512 B**
64 KiB	0,37-0,40	0,39	0,43	0,83	1,22	1,18	1,60
128 KiB	0,37-0,38	0,40	0,41	0,78	1,17	1,23	1,66

(ns/update, mediana, valores de `e1/L2/cache_table.md`)

Estimativa: entre 64 B e 128 B. Os tempos não permitem identificar um valor único de forma fiável.

Nos dois tamanhos, o tempo mantém-se praticamente constante até 32 B e depois sobe duas vezes seguidas: de 32 para 64 B (cerca de 1,9 vezes) e de 64 para 128 B (cerca de 1,5 vezes). Entre 128 e 256 B forma-se um pequeno patamar. O padrão ideal seria uma subida até ao tamanho da linha seguida de um patamar. Aqui há dois degraus em vez de um, e os tempos não permitem dizer com certeza qual deles corresponde ao tamanho da linha. O primeiro e maior aumento relativo ocorre de 32 para 64 B. A partir de 512 B o tempo volta a subir, por efeitos que esta experiência não permite isolar. Por isso, os tempos indicam uma linha entre 64 e 128 B.

Exercício 2: Comportamento da cache com contadores de hardware

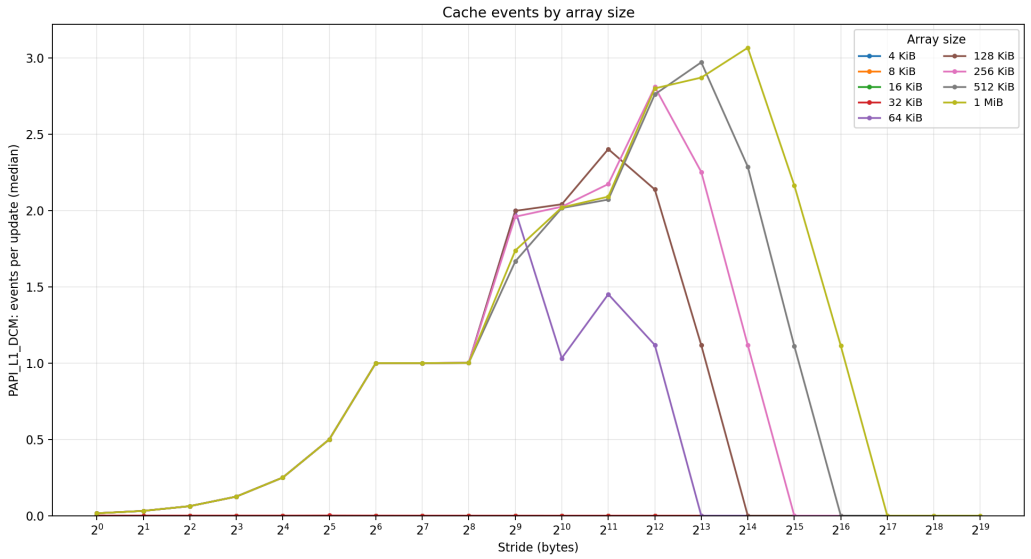
a) Plataforma e eventos

Usou-se a mesma máquina do Exercício 1 (Intel Core i5-10500, Linux 6.12.40, GCC 14.2.1), com PAPI 7.2.0 compilado localmente. Opções de compilação: `-O2 -std=c11 -Wall -Wextra`, com ligação estática a `libpapi.a` e `-ldl -lpthread`.

Experiência	Evento usado	Evento(s) nativo(s)	Descrição (`event_details.txt`)
L1	`PAPI_L1_DCM` (Level 1 data cache misses)	`L1D:REPLACEMENT`	"L1D cache, masks:L1D Data line replacements"
L2	`PAPI_L2_DCM` (Level 2 data cache misses)	`LLC_REFERENCES` menos `L2_RQSTS:CODE_RD_MISS`	`LLC_REFERENCES`: "count each request originating from the core to reference a cache line in the last level cache. The count may include speculation, but excludes cache line fills due to hardware prefetch" e `L2_RQSTS:CODE_RD_MISS`: "L2 requests, masks:L2 cache misses when fetching instructions"

b) Capacidade da L1 (`PAPI_L1_DCM`)

Intervalo final: 4 KiB a 1 MiB (o intervalo por omissão já cobria a transição).



Estimativa: entre 32 KiB e 64 KiB (intervalo [32, 64[KiB).

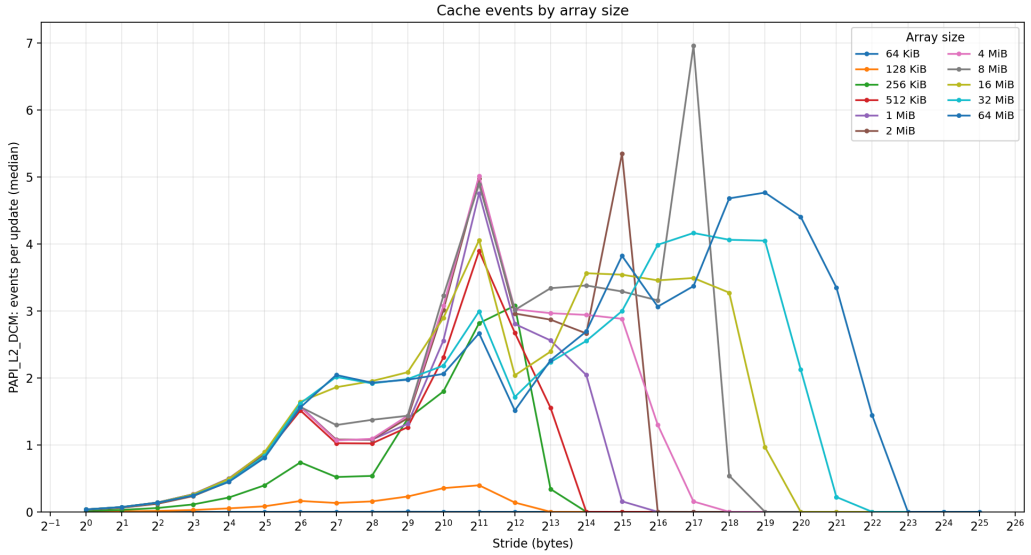
Stride \ Array	4 KiB	8 KiB	16 KiB	32 KiB	64 KiB	128 KiB
1 B	8,6e-6	4,1e-5	2,2e-5	4,8e-4	0,0156	0,0156
16 B	9,5e-7	5,7e-6	7,6e-6	4,8e-4	0,250	0,250
64 B	1,3e-5	4,6e-5	9,5e-6	4,8e-4	1,000	1,000
256 B	9,5e-7	1,2e-5	3,1e-5	1,2e-4	1,001	1,001

(eventos por atualização, mediana, valores de `e2/programs/cm1/L1/cache_table.md`)

Até 32 KiB há praticamente zero eventos (no máximo 0,001 por atualização): depois do aquecimento, todos os acessos são hits. Com 64 KiB, a taxa passa para exatamente 1 miss por linha acedida, o que indica que o array não cabe na L1 (quando o programa volta ao início, as primeiras linhas já foram substituídas pelas últimas, e falham todas outra vez).

c) Capacidade da L2 (`PAPI_L2_DCM`)

Intervalo final: 64 KiB a 64 MiB. O mínimo é o dobro de 32 KiB. O intervalo é o mesmo da alínea 1d), que já tinha mostrado a transição perto de 256 KiB.



Estimativa: cerca de 0,25 MiB (intervalo [0,125; 0,5[MiB).

Stride \ Array	64 KiB	128 KiB	256 KiB	512 KiB	1 MiB	2 MiB
1 B	7,9e-5	0,0038	0,0158	0,0348	0,0355	0,0303
16 B	4,8e-5	0,052	0,214	0,489	0,499	0,453
64 B	2,8e-4	0,162	0,738	1,512	1,573	1,561
128 B	4,5e-4	0,132	0,520	1,024	1,077	1,078

(eventos por atualização, mediana, valores de `e2/programs/cm1/L2/cache_table.md`)

- Com 64 KiB praticamente não há falhas na L2 (valores muito abaixo de 0,01).
- Com 128 KiB, os valores são cerca de 10% dos obtidos com 1 MiB.
- Com 256 KiB, sobem para cerca de 45% dos obtidos com 1 MiB.

- A partir de 512 KiB, os valores coincidem com os dos arrays maiores (95 a 98% dos de 1 MiB).

A transição ocorre portanto entre 128 KiB e 512 KiB, com o array de 256 KiB a meio, tal como observado na alínea 1d). O intervalo cobre os dois lados da transição.

d) Tamanho da linha da L1

Array escolhido: 128 KiB (maior que a L1 estimada, [32, 64[KiB, e menor que a L2 estimada, cerca de 0,25 MiB). O mesmo comportamento observa-se no array de 64 KiB.

Array \ Stride s (B)	**1**	**2**	**4**	**8**	**16**	**32**	**64**	**128**	**256**
64 KiB	0,01565	0,0313	0,0625	0,125	0,250	0,500	1,000	1,000	1,001
128 KiB	0,01565	0,0313	0,0625	0,125	0,250	0,500	1,000	1,000	1,001

(eventos por atualização, mediana, valores de `e2/programs/cm1/L1/cache_table.md`)

Estimativa: 64 B.

Se a linha tiver 64 B, com um stride s menor que 64 B há 64/s atualizações em cada linha, e só a primeira falha. Prevê-se assim s/64 misses por atualização: 1/64 = 0,0156 no stride 1, 16/64 = 0,25 no stride 16 e 32/64 = 0,5 no stride 32. Com s igual ou superior a 64 B, cada atualização acede a uma linha diferente, e prevê-se 1 miss por atualização.

Os valores medidos coincidem com esta previsão em todos os strides da tabela: duplicam a cada duplicação do stride até 64 B e ficam em 1,0 a partir daí. Uma linha de tamanho diferente daria valores diferentes, portanto podemos concluir que o tamanho da linha é 64 B.

e) Justificação e comparação com o Exercício 1

Capacidade da L1: Com um stride de 64 B, o array de 32 KiB dá 4,8e-4 eventos por atualização e o de 64 KiB dá 1,00. Nos tempos, 0,38 e 0,82 ns. Os dois métodos apontam para o intervalo [32, 64[KiB.

Capacidade da L2: Com um stride de 64 B, os arrays de 128, 256 e 512 KiB dão 0,16, 0,74 e 1,51 eventos por atualização, e 0,78, 1,10 e 1,26 ns. Os dois métodos situam a transição perto de 256 KiB e estimam cerca de 0,25 MiB. A única discordância está no array de 128 KiB: os contadores já registam cerca de 10% dos eventos dos arrays grandes, mas nos tempos não se distingue do de 64 KiB (0,78 contra 0,83 ns).

Tamanho da linha: Os contadores dão 64 B sem ambiguidade (alínea d), enquanto os tempos só permitiam situá-lo entre 64 e 128 B.

Os contadores tornaram as transições mais claras. Nos tempos, as transições são aumentos de cerca de 1,5 a 2 vezes. O tempo medido inclui também o custo do próprio ciclo e, como refere o enunciado, a latência de um miss pode sobrepor-se a outro trabalho. Nos contadores, o miss é contado mesmo nesse caso, e os valores passam de praticamente zero para valores da ordem de 1 (várias ordens de grandeza), com pouca variação entre amostras.

Limitações:

- Na L2, os eventos não correspondem a um miss por linha (cerca de 1,5 por atualização com um stride de 64 B nos arrays grandes), e as experiências não permitem identificar a origem dos eventos adicionais. Por isso, a capacidade da L2 foi estimada comparando os tamanhos de array entre si, e não pelo valor absoluto. Na L1, os valores também ultrapassam 1 com strides de 512 B ou mais, mas estes pontos não foram usados nas estimativas.
- Com 128 KiB e 256 KiB ocorre apenas parte dos eventos (cerca de 10% e 45%), pelo que não é possível identificar o valor exato da capacidade da L2, e por isso se apresenta um intervalo.
- `PAPI_L2_DCM` resulta da diferença entre dois contadores, e algumas execuções foram rejeitadas por produzirem valores negativos, pelo que foi necessário repetir a experiência.

Exercício 3: Desenho da cache com o Ripes

Simulador: Ripes v2.2.6, com o processador de ciclo único de 32 bits e o programa `workload.asm` fornecido. Foi alterada apenas a configuração da cache de dados. Cada configuração foi executada uma vez, desde o início, com a cache vazia e as estatísticas a zero.

Em todas as tabelas, S é o número de sets, A o número de vias e W o número de palavras de 4 bytes por linha. A capacidade de dados é o número de linhas (S vezes A) multiplicado pelo tamanho de cada linha (W vezes 4 bytes). Em todas as execuções, hits + misses = 512, o número total de acessos do programa (256 escritas no primeiro ciclo, 128 leituras e 128 escritas no segundo).

a) Comparação de presets

Os três presets usam write-back (write hit), write-allocate (write miss) e substituição LRU.

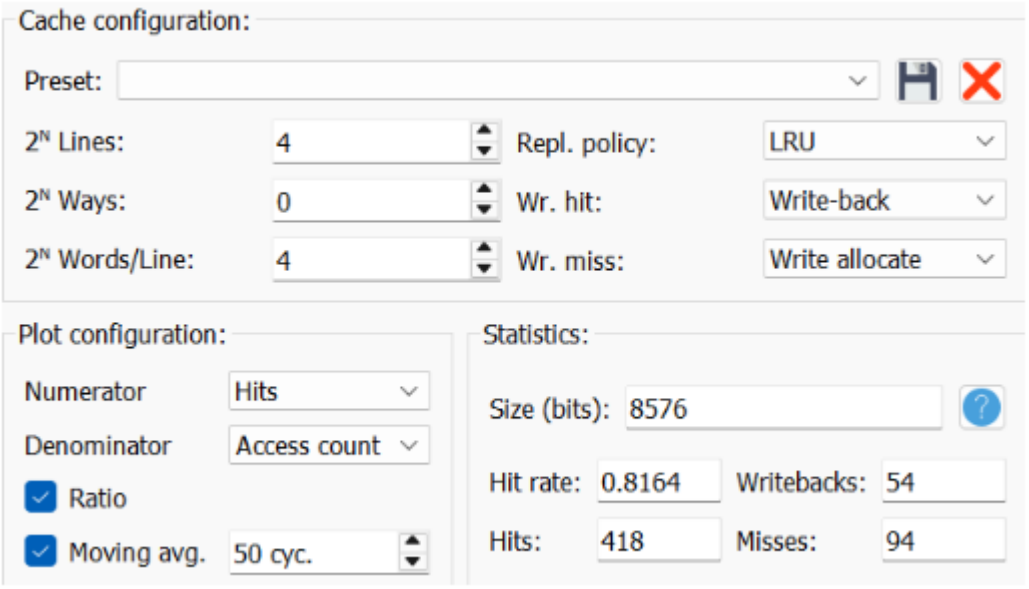
Preset	S	A	W	Capacidade	Hits	Misses	Writebacks	Hit rate
32-entry 4-word direct-mapped	32	1	4	512 B	120	392	315	23,44%
32-entry 4-word fully associative	1	32	4	512 B	320	192	116	62,50%
32-entry 4-word 2-way set associative	16	2	4	512 B	28	484	360	5,47%

b) Seleção de uma configuração

Todas as configurações usam write-back, write-allocate e substituição LRU, e ocupam a totalidade do orçamento de 1024 bytes (8192 bits) de dados. Foram testadas configurações de mapeamento direto (A = 1) e associativas de 2 vias (A = 2), com linhas de 2, 4, 8, 16 e 32 palavras (8 a 128 bytes).

#	S	A	W	Capacidade	Hits	Misses	Writebacks	Hit rate
1	128	1	2	1024 B	267	245	95	52,15%
2	64	1	4	1024 B	346	166	76	67,58%
3	32	1	8	1024 B	414	98	48	80,86%
4	16	1	16	1024 B	418	94	54	81,64%
5	8	1	32	1024 B	378	134	85	73,83%
6	64	2	2	1024 B	255	257	107	49,80%
7	32	2	4	1024 B	311	201	111	60,74%
8	16	2	8	1024 B	395	117	69	77,15%
9	8	2	16	1024 B	407	105	73	79,49%
10	4	2	32	1024 B	371	141	110	72,46%

A configuração com menos misses é a 4 (S = 16, A = 1, W = 16), com 94 misses. Não há empates.



c) Justificação

A configuração escolhida (S = 16, A = 1, W = 16: mapeamento direto, linhas de 64 bytes) teve o menor número de misses (94). Os arrays são percorridos por elementos consecutivos (ou, no caso de d, de 3 em 3), pelo que linhas maiores aproveitam a localidade espacial: no mapeamento direto, os misses descem de 245 com linhas de 8 bytes para 94 com linhas de 64 bytes. Com linhas de 64 bytes, cada miss traz 16 inteiros consecutivos, que servem os acessos seguintes enquanto a linha não for substituída.

O array d é acedido com um salto de 12 bytes, por isso uma linha de 64 bytes ainda serve cerca de 5 dos seus acessos (64 / 12 = 5,3).

Em contraste, a configuração 1 (W = 2, linhas de 8 bytes) só guarda 2 inteiros por linha, e o salto de 12 bytes de d é maior do que a linha. Assim, cada acesso a d usa uma linha diferente, e nenhum aproveita a linha trazida pelo acesso anterior.

Nas configurações com mapeamento direto, os misses descem de 245 para 166, 98 e 94 quando W passa de 2 para 4, 8 e 16. Em W = 32 a tendência inverte-se (134 misses). O programa usa cerca de 1,5 KiB de endereços (768 bytes de a, b e c, e cerca de 760 bytes percorridos em d), mais do que cabe na cache, mas isso é igual em todas as configurações testadas e não explica, por si só, a inversão. A explicação provável são os seguintes conflitos:

- No mapeamento direto, cada endereço só pode ir para uma linha da cache, que se calcula como "endereço módulo 1024 bytes". A parte final de d (endereços acima de 1024 a contar do início de a) cai nas mesmas linhas que a e b e expulsa-as.
- Com linhas de 128 bytes, a cache tem só 8 linhas, cada uma disputada por mais blocos, e cada linha expulsa por conflito leva consigo mais dados que ainda iam ser usados.

O simulador não classifica os misses por tipo. Esta explicação é por isso uma interpretação do padrão de acessos e não uma medição direta.

Com 2 vias, a associatividade não produziu benefício: estas configurações foram piores do que as equivalentes com mapeamento direto em todos os tamanhos de linha testados (257 contra 245, 201 contra 166, 117 contra 98, 105 contra 94 e 141 contra 134). Cada endereço pode ser colocado numa de 2 linhas do seu set, o que ajuda a evitar conflitos, mas, com a capacidade fixa, o número de sets passa para metade e cada via cobre apenas 512 bytes de endereços. Assim, endereços separados por múltiplos de 512 bytes caem no mesmo set, e formam-se dois grupos que disputam os mesmos sets: o primeiro com a, c e uma parte do final de d, e o segundo com b, o início de d e o resto do final de d. Em ambos os casos há três blocos para apenas 2 vias.

Na alínea b) não foram testadas configurações com mais de 2 vias. No entanto, os presets da alínea a), com 512 bytes, mostram o mesmo padrão e sugerem que uma associatividade maior compensaria: a cache de 2 vias teve mais misses do que a de mapeamento direto (484 contra 392), mas a totalmente associativa teve muito menos (192).

A mesma configuração não seria necessariamente a melhor para qualquer programa com o mesmo tamanho total de arrays. A melhor configuração depende do padrão de acessos, e não só da quantidade de dados:

- Num programa com acessos muito espaçados ou aleatórios, apenas uma pequena parte de cada linha seria utilizada, pelo que linhas mais curtas seriam preferíveis.
- Num programa cujos arrays ocupassem endereços mapeados nos mesmos sets (por exemplo, separados exatamente pelo tamanho da cache), uma cache de mapeamento direto sofreria conflitos constantes, e uma maior associatividade traria um benefício significativo.
- Num programa que reutilizasse repetidamente uma pequena zona dos dados, essa zona caberia na cache em quase todas as configurações, pelo que a escolha da configuração teria um impacto reduzido.